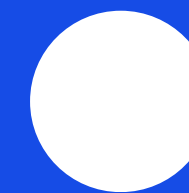


Codeit Sprint [AI] 초급 프로젝트

경구약제 이미지 인식

PillaTech



안수진, 김한별, 원숙현, 황예원

Fail Fast

Fail Fast

라이브러리 버전 고정

학습과 추론 설정
yaml파일로 일원화

재현성 문제...

seed, deterministic 고정

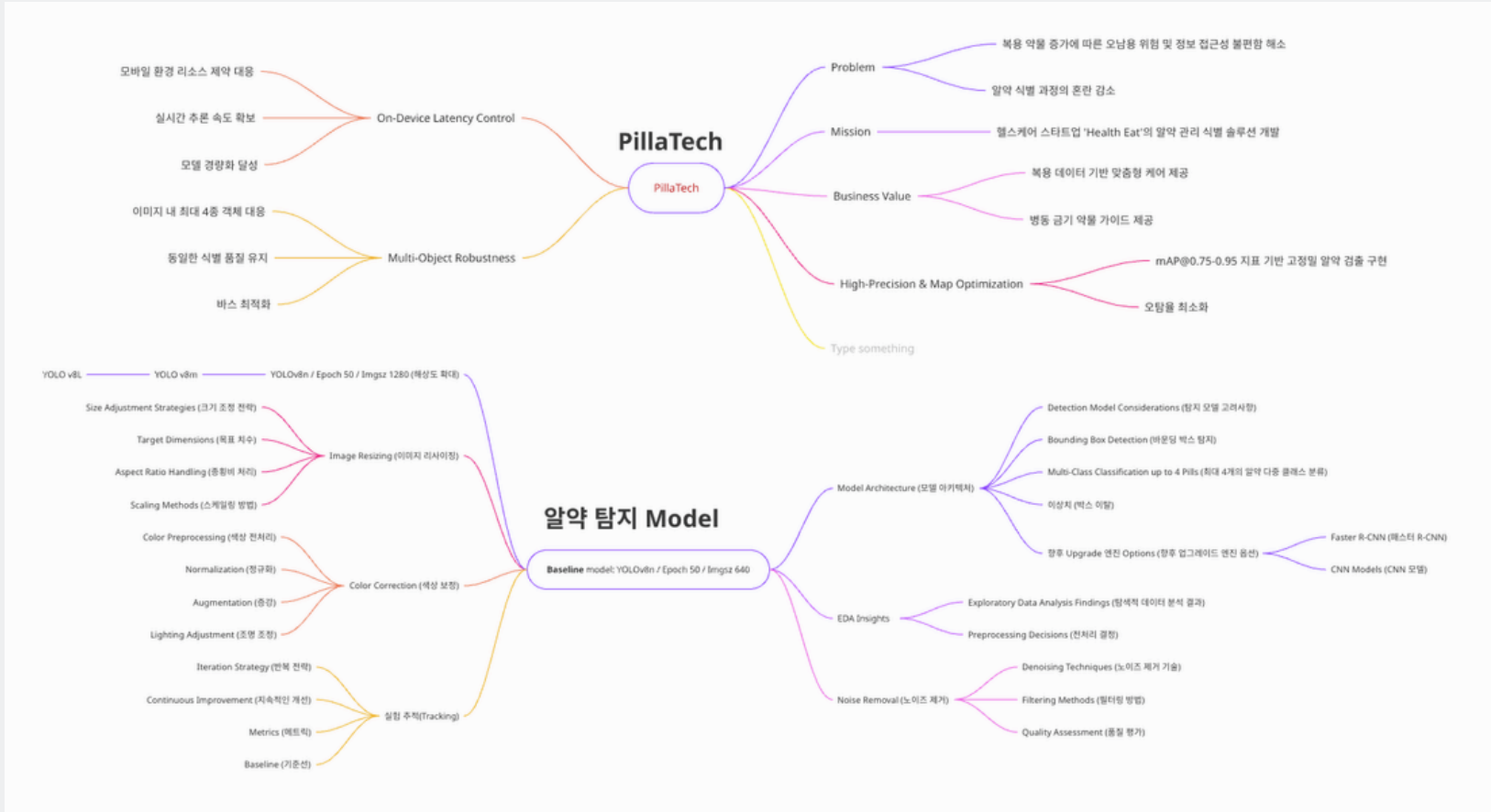
Train/test 분할방식
복합변수로 비교검증 불가!

해상도, 배치사이즈, 하드웨어 등의 사이즈 축소

“TTA를 넣으면 반드시 성능이 좋아질 것이다.”
라는 가설이 틀릴 수가 있다니...

자동 설정된 Adam 외에도 더 최적의
Optimizer를 찾아낼 수 있을까?

30	Hanbyeol_base_yolov11m.csv		한별_exp1_YOLOv11m	적용	50	계승적 분할	YOLO v11m	640	Yolo 기본증강(?)	-	0.88479	3/30 한별: yolov11 medium 결과입니다						
30	Hanbyeol_base_yolov11l.csv		한별_exp1_YOLOv11L	적용	50	계승적 분할	YOLO v11L	640	Yolo 기본증강(?)	-	0.88829	30분 한별: yolov11 large 결과입니다						
				적용	50	계승적 분할	YOLOv11X	640	Yolo 기본증강	-	5시간 예상							
24	hanbyeol_v4_yolov8s_results_team04_06.csv	v4_yolov8s_results_team04_06.csv	한별_v4_2	적용	50	계승적 분할	Yolo v8s	640	최귀 클래스 증강 (Copy Paste)	-	0.89687	260324 mps 학습, 최귀 클래스 Copy Paste 증강 5시간 학습 완료	0.9112	0.9117	-			
24	hanbyeol_v4_tta	v4_yolov8s_results_team04_06.csv	한별_v4_2	적용	50	계승적 분할	Yolo v8s	640	최귀 클래스 증강 (Copy Paste)	TTA	0.90361 (50에 폭 전 약 10에폭 결과)	최귀 증강 + v8n vs 최귀 증강 + v8s (50에폭을 다 돌리지 못해서 성능이 떨어진 것 같다)	0.98	0.97	-			
26	hanbyeol_v5_with_tta	hanbyeol_v5_with_tta.csv	한별_v5	적용	50	계승적 분할	yolov8s	1024	최귀 클래스 증강(Copy Paste) + 실시간 온라인 증강 (mosaic=1.0, mixup=0.1, scale=0.5, label_smoothing=0.1)	TTA	0.94569	최귀 클래스 증강 + 고해상도(imgsz:1024) + 실시간 학습 증강 + yolov8s + tta	0.98	0.97	-			



코드잇_4팀 / ... / 아이디어노트 / Experiment Log											2분 전 편집							
	개글 실제 파일명	개글 실제 파일명	개인(이름) 표기	고정 변수	고정 변수	고정 변수	주요 변수	주요 변수	주요 변수	주요 변수	Score	비고	기타 변수	기타 변수	기타 변수	기타 변수	기타변수	기타변수
날짜	개글 실제 파일명(영문이나실_v1_지난 버전에서 update 된 변수)	개글 실제...파일명 (0325까지)	각자의 실험로그_이번 실험에서 추가된 변수	Data cleaning 여부 (9 json: 8 missing box, 1 invalid box)	Epoch (early stop 사정)	Train/test 분할방식	Model	해상도 imgsiz	Data 증강 적용방식	csv변환시 TTA 적용여부 (Test Time Aug)	개글 Score	비고(개인적 고찰, 사전, 참고사항)	Best Epoch	mAP50	mAP@50-95	mAP@75	정밀도 (Precision)	재현율 (Recall)
23	예환님 배포한 원래 모델		YOLO v8n	적용	50	랜덤 분할	YOLO v8n	640	Yolo 기본증강	-				0.79	0.76			
23	Basemodel 1.0	basemodel_results_PillaTech_team04_02.csv	YOLO v8n (Baseline)	Data cleaned	50	Stratified Split (계층적 분할)	Baseline: YOLO v8n	640	Yolo 기본증강	-	0.70166	수진_Exp 1-2: Stratified (Full)와 똑같은 실험인데 개글 점수와 차이가 나는게 이상함	49	0.90224	0.87815		측정 실패	
27	hanbyeol_exp1_8s		한별_exp1_YOLOv8s	적용	50	계층적 분할	YOLO v8s	640	Yolo 기본증강	-		베이스모델과 비교 (yolov8n vs yolov8s)		0.99284	0.98331			
27	hanbyeol_exp1_8m		한별_exp1_YOLOv8m	적용	50	계층적 분할	YOLO v8m	640	Yolo 기본증강	-		베이스모델과 비교 (yolov8n vs yolov8m)		0.99279	0.9845			
28	Soog_yolov8l_v5_local.csv		수현_v5_YOLOv8l	적용	50	계층적 분할	YOLO v8l	640	Yolo 기본증강	-	0.92471	베이스모델과 비교 (yolov8n vs yolov8l): imgsiz = 640, Epoch = 50, 계층적 분할, 배치는 4이고 TTA 적용 안한 baseline 1.0 용 test.py(이미지 역매칭기법) 사용						
29			한별_exp1_YOLOv11n	적용	50	계층적 분할	YOLO v11n	640	Yolo 기본증강(?)	-								
29			한별_exp1_YOLOv11s	적용	50	계층적 분할	YOLO v11s	640	Yolo 기본증강(?)	-								
30	Hanbyeol_base_yolov11m.csv		한별_exp1_YOLOv11m	적용	50	계층적 분할	YOLO v11m	640	Yolo 기본증강(?)	-	0.88479	3/30 한별: yolov11 medium 결과입니다						
30	Hanbyeol_base_yolov11l.csv		한별_exp1_YOLOv11L	적용	50	계층적 분할	YOLO v11L	640	Yolo 기본증강(?)	-	0.89829	3/30 한별: yolov11 Large 모델 학습 결과입니다. yolov11 Medium 모델보다 높은 성능이 나와서 정확히 개글 점수로 비교해보고 싶습니다						
				적용	50	계층적 분할	YOLOv11X	640	Yolo 기본증강	-		5시간 예상						
24	hanbyeol_v2_copy_paste	v3_augmented_results_PillaTech_team04_03.csv	한별_exp2_copy_paste	적용	50	계층적 분할	Yolo v8n	640	최귀 클래스 증강 (Copy Paste)	-	0.89682	260324 mps로 학습, 최귀 클래스 Copy-Paste 증강 후 모델 학습 완료	-	0.97942	0.95175	-		
24	hanbyeol_v4_yolov8s	v4_yolov8s_results_PillaTech_team04_04.csv	한별_exp2_YOLOv8s	적용	50	계층적 분할	Yolo v8s	640	최귀 클래스 증강 (Copy Paste)	-	0.83256 (50에 폭 전 약10배폭 결과)	최귀 증강 + v8n vs 최귀 증강 + v8s (50에 폭을 다 돌리지 못해서 성능이 떨어진 것 같다)		0.98	0.97	-		
24	hanbyeol_v4_tta	v4_yolov8s_results_tta_team04_06.csv	한별_v4_2	적용	50	계층적 분할	Yolo v8s	640	최귀 클래스 증강 (Copy Paste)	TTA	0.90361 (50에 폭 전 약 10배폭 결과)	최귀 증강 + v8s + TTA (50에폭을 다 돌리고 했더라면 더 높은 점수가 나왔으리라 예상)				-		
26	hanbyeol_v5_with_tta	hanbyeol_v5_with_tta.csv	한별_v5	적용	50	계층적 분할	yolov8s	1024	최귀 클래스 증강(Copy Paste) + 실시간 온라인 증강 (mosaic=1.0, mixup=0.1, scale=0.5, label_smoothing=0.1)	TTA	0.94569	최귀 클래스 증가 + 고해상도(imgsz:1024) + 실시간 학습 증강 + yolov8s + tta		0.98	0.97			
30	hanbyeol_copy_box_cls_default		한별_box&cls_default	적용	50	계층적 분할	yolo v11s	640	최귀 클래스 증강(Copy Paste v2)		0.87052	box → default: 7.5, cls → default:0.5		0.97716	0.96439		0.95605	0.97826
30	hanbyeol_box12.0_cls0.5		한별_box&cls (12.0, 0.5)	적용	50	계층적 분할	yolo v11s	640	최귀 클래스 증강(Copy Paste v2)			box → 12.0, cls → 0.5 (bbox를 더 정교하게 그리도록 수정)		0.9711	0.9594	0.9594	0.9142	0.9798
			수진_Exp 20H3:yolo11s_hsvh020				Yolo v11s											
30	Yewon_v5_yolov11s_CLAHE.csv		예환: Base2+ CLAHE		167		Yolo v11s		CLAHE		0.95902	3/30 예환: Yewon_v4_yolov11s.csv에서 CLAHE 적용						
31	Hanbyeol_hsv_v_0.4.csv		한별: HSV_V default 0.4		87		Yolo v11s		HSV_V default 0.4		0.93634	3/31 한별: HSV_V default 0.4결과, 타당성 검증됨	0.9334	0.9785	0.9781	0.9707		0.4
31	Hanbyeol_hsv_v_0.6.csv		한별: hsv_v default 0.6		87		Yolo v11s		hsv_v default 0.6		0.92808	3/31 한별: hsv_v default 0.6결과, 타당성 검증됨	0.9551	0.9783	0.9822	0.9741		0.6
31	Soog_v8_B2_yolo11s_purelySGD.csv		수현_Yolo 11s_SGD_v2		50	계층 분할 + 랜덤 최귀	Yolo v11s	960		미적용 (custom-수진버전)	0.60910	3/31 수현: Baseline2.0_수진 exp15)에서 오토 파라미터 튜닝, 오토마이즈한 SGD로 변경, 가중치 best.py 별도학습						
31	Soog_v8_B2_yolo11s_SGD.csv		수현_Yolo 11s_SGD_v1		50	계층 분할 + 랜덤 최귀	Yolo v11s	960		미적용 (custom-수진버전)	0.94536	Baseline2.0_수진 exp15)에서 오토마이즈한 SGD로 변경 (수진님 공유 가중치 best.py 파일, batchsize = 16, workers = 8으로 복구)						
31	Sujin_v19A_yolo11s_res960_no_hsvh.csv		수진_Exp 20H4:yolo11s_hsvh030		16	계층 분할 + 랜덤 최귀	Yolo v11s	960	exp15 + hsv_h 효과 0.000		0.96678	3/31 수진: exp15 + hsv_h 효과 0.000으로 모두 제거						
31	Sujin_v20H3_yolo11s_res960_hsvh020.csv		수진_Exp 20H7:yolo11s_hsvh025		16	계층 분할 + 랜덤 최귀	Yolo v11s	960	exp15 + hsv_h 효과 0.20		0.97154	3/31 수진: exp15 + hsv_h 효과 0.20으로 상한선 검증됨	0.9548	0.9801	0.9931	0.9914	0.20	
31	Sujin_v21M2_yolo11s_res960_mosaic06.csv		수진_Exp 21M0:yolo11s_mosaic00		16	계층 분할 + 랜덤 최귀	Yolo v11s	960	mosaic 0.6		0.96154	3/31 수진: mosaic 0.6 비교 테스트						
31	Sujin_v21M0_yolo11s_res960_mosaic00.csv		수진_Exp 21M2:yolo11s_mosaic00		16	계층 분할 + 랜덤 최귀	Yolo v11s	960	mosaic 0.0		0.96894	3/31 수진: mosaic 0.0 비교 테스트						
1	Sujin_v22H3M0_yolo11s_res960_hsvh020_mosaic00.csv		수진_Exp 22_hsvh=0.020 + mosiar 0.00		16	계층 분할 + 랜덤 최귀	Yolo v11s	960	hsvh=0.020 + mosiar 0.00		0.97199	4/1 수진: hsvh=0.020 + mosiar 0.00 변수 추가 조합	0.9753	0.9630	0.9935	0.9915		
1	Sujin_v23C1_yolo11s_res960_hsvh020_mosaic00_clahe15.csv			적용 (확인)	16	계층 분할 + 랜덤 최귀	Yolo v11s	960	exp12 + clahe 15 추가	미적용	0.96435	4/1 수진 : exp12 + clahe 15 추가	0.9683	0.9643	0.9948	0.9941	0.15	
날짜	개글 실제 파일명(영문이나실_v1_지난 버전에서 update 된 변수)	개글 실제...파일명 (0325까지)	각자의 실험로그_이번 실험에서 추가된 변수	Data cleaning 여부 (9 json: 8 missing box, 1 invalid box)	Epoch (early stop 사정)	Train/test 분할방식	Model	해상도 imgsiz	Data 증강 적용방식	csv변환시 TTA 적용여부 (Test Time Aug)	개글 Score	비고(개인적 고찰, 사전, 참고사항)	Precision	Recall	mAP50	mAP@50-95	clahe	hsv_v(0.4)
	개글 실제 파일명	개글 실제 파일명	개인(이름) 표기	고정 변수	주요 변수	고정 변수	고정 변수	주요 변수	주요 변수	주요 변수	Score	비고	기타 변수	기타 변수	기타 변수	기타 변수		
+																		
보고서 제출 정리음																		

EDA

데이터셋 정의 및 구조

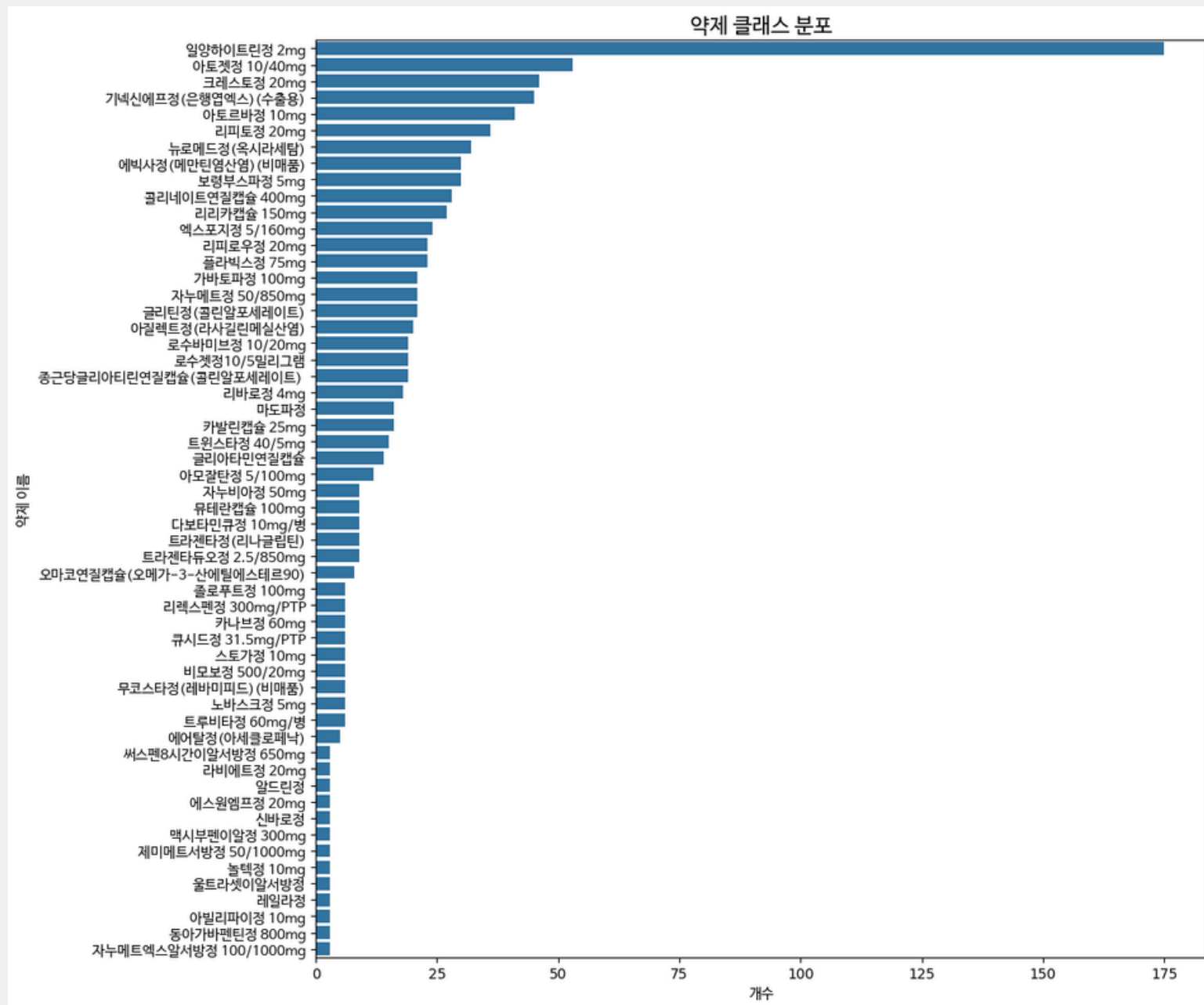
1. 데이터셋 개요 (Overview)

- Source: **AI hub 경구약제 이미지** 데이터셋 (코드잇에서 가공 후 제공)
- Scale: 총 -000장의 이미지 / 000종의 알약 클래스

2. 원본 데이터 구조 (Raw Data Structure)



탐색적 데이터 분석 (EDA Insight) – 클래스 분포



- 특정 클래스 편중 및 최소 데이터(3개) 클래스 다수 존재
- 전략

1. Sampling: **Stratified Split**으로 학습/검증셋 균형 확보

2. Augmentation: 데이터 부족 클래스 대상
Copy-Paste 및 **Oversampling** 적용

탐색적 데이터 분석 (EDA Insight) – 무결성

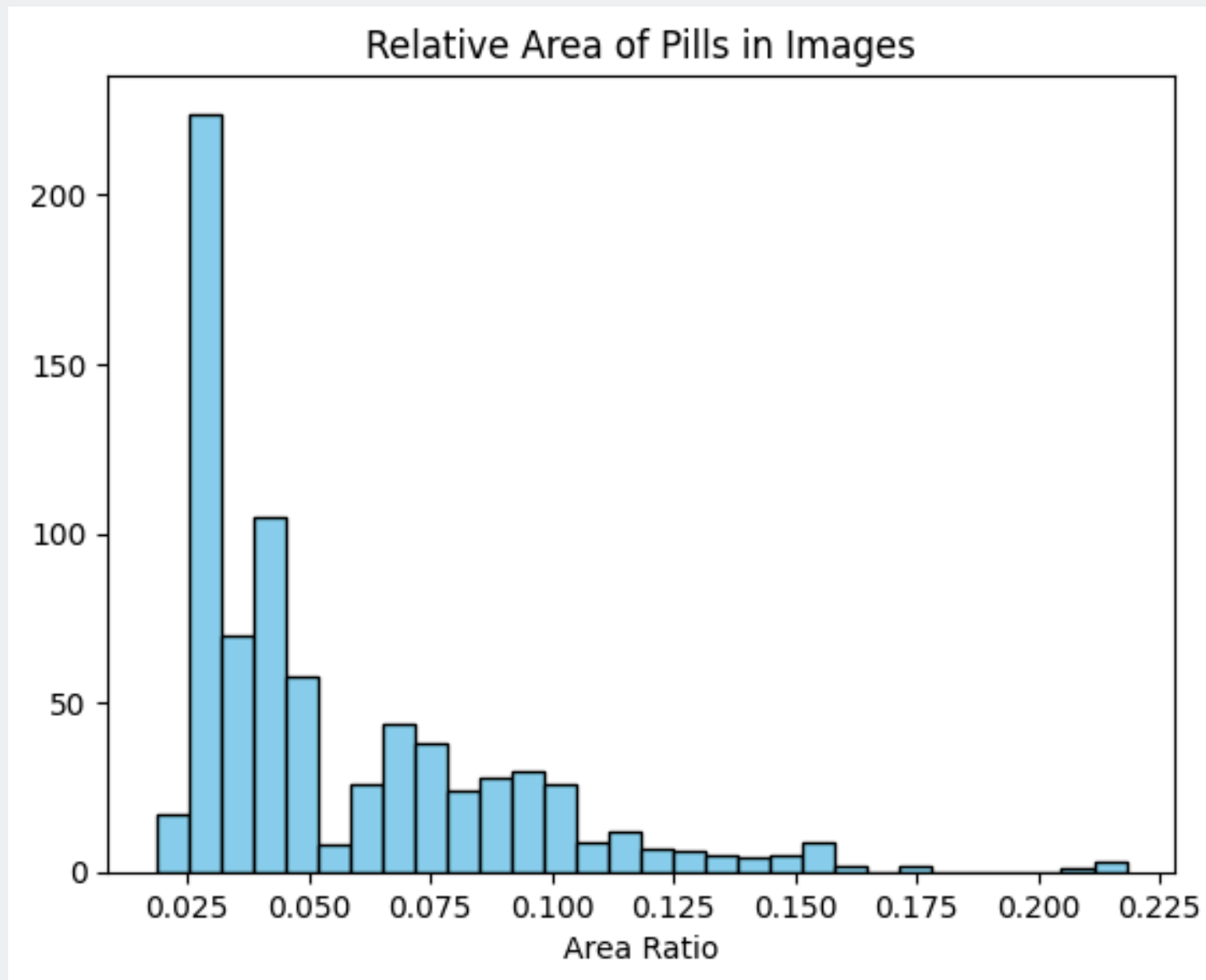


- Invalid Box 1건 및 Missing Box 8건
- 전략

데이터 가치 보존을 위해 삭제 대신 **직접 라벨링** 수행

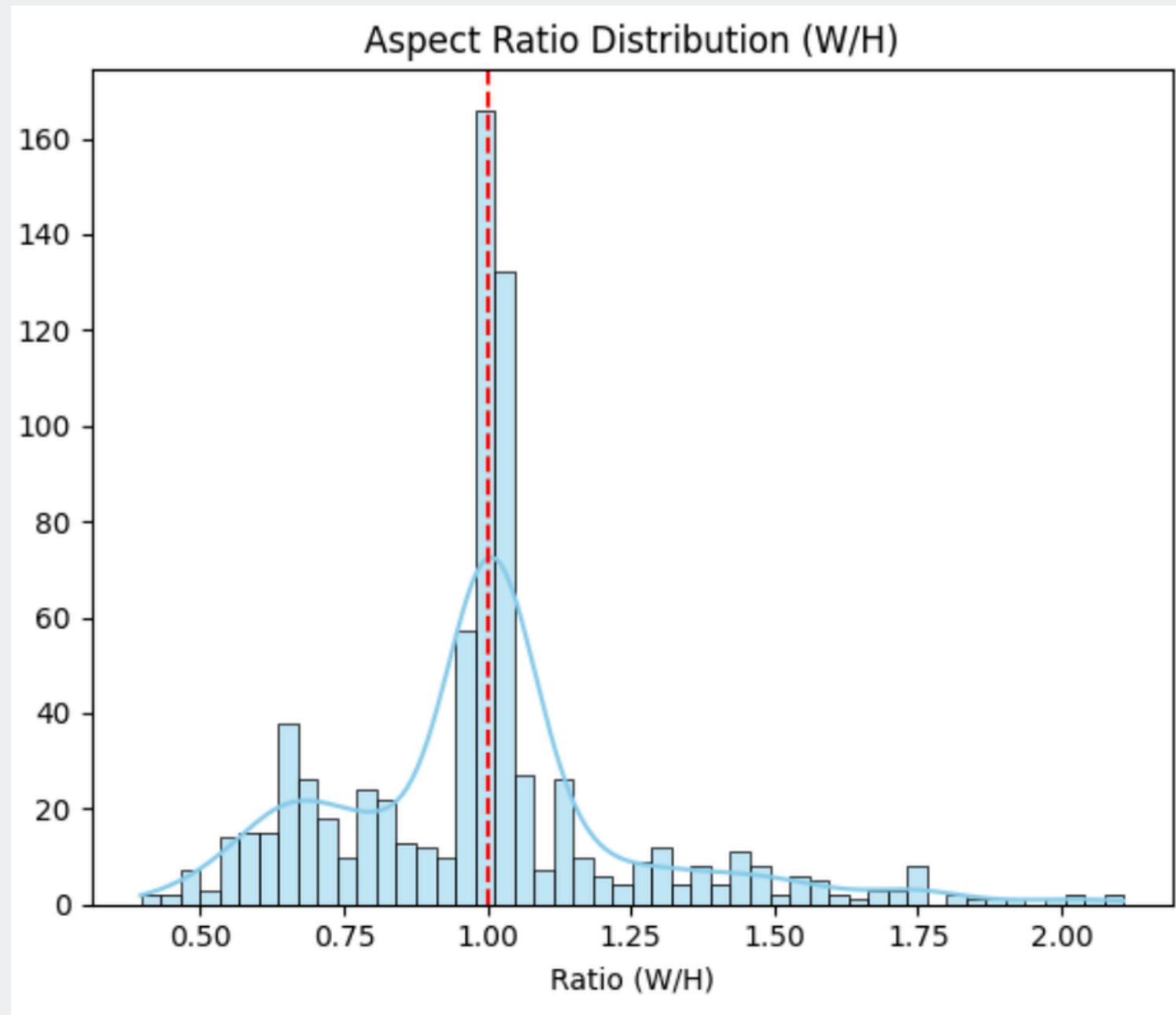
- **좌표값 산출**: 이미지 내 객체 경계면을 기준으로 좌표값 정밀 추출
- **JSON 구조화**: 해당 이미지의 어노테이션 JSON 파일을 직접 수정하여 정규화된 좌표값 업데이트

탐색적 데이터 분석 (EDA Insight) – 객체 크기



- 이미지 대비 객체 면적 비율이 평균 약 5.6% 수준으로 낮음
- 전략
 1. 정보 손실 방지를 위한 **고해상도(960-1080px)** 정하기
 2. **Mosaic** Augmentation 적용

탐색적 데이터 분석 (EDA Insight) – 객체 모양



- 알약 이미지 특성상 형태가 매우 유사하여 변별력 확보 필요
- 장방형 비율이 존재
- 전략

객체 식별력 강화(또렷한 이미지)를 위한 **CLAHE** 기법 도입
다양한 배치 각도에 대한 검출력을 위해 **Rotation/Flip**

베이스라인 모델 선정

아키텍처 탐색 및 모델 선정

1. 후보 모델 비교

Faster R-CNN

2-Stage Detector
높은 검출 정밀도
추론 속도가 느림

YOLO

1-Stage Detector
속도와 정확도의 최적 밸런스
모바일 환경 최적화 용이

RT-DETR

Transformer
뛰어난 성능
높은 학습 리소스 소모

2. YOLO 선정 근거

실시간성

헬스케어 앱의 특성상 즉각적인 알약 식별을 위한 빠른 추론 속도 확보 가능

효율성

적은 파라미터로도 높은 성능을 내며, 빠른 실험 반복에 최적화

확장성

앞선 EDA 전략(Mosaic, Augmentation)을 기본 지원하며 온디바이스 배포에 유리

베이스라인 1.0

1. Baseline 1.0 개요

- 모델 구조: **YOLOv8n**
- 주요 설정:
 - 입력 해상도: 640
 - 전처리: 기본 이미지 (Raw Data)
 - 데이터 분할: Stratified Split (계층적 분할)
 - 증강(Augmentation): 기본 설정
- 핵심 지표: mAP50 기준 약 0.90224, 캐글 점수 약 0.70166

베이스라인 1.0

2. Baseline 선정 이유

(1) 데이터 본연의 특징 파악

원본 데이터셋이 가진 객체(약제)의 특징과 검출 난이도를 객관적으로 파악하기 위한 기준으로 설정함.

(2) 모델 성능의 하한선 확보

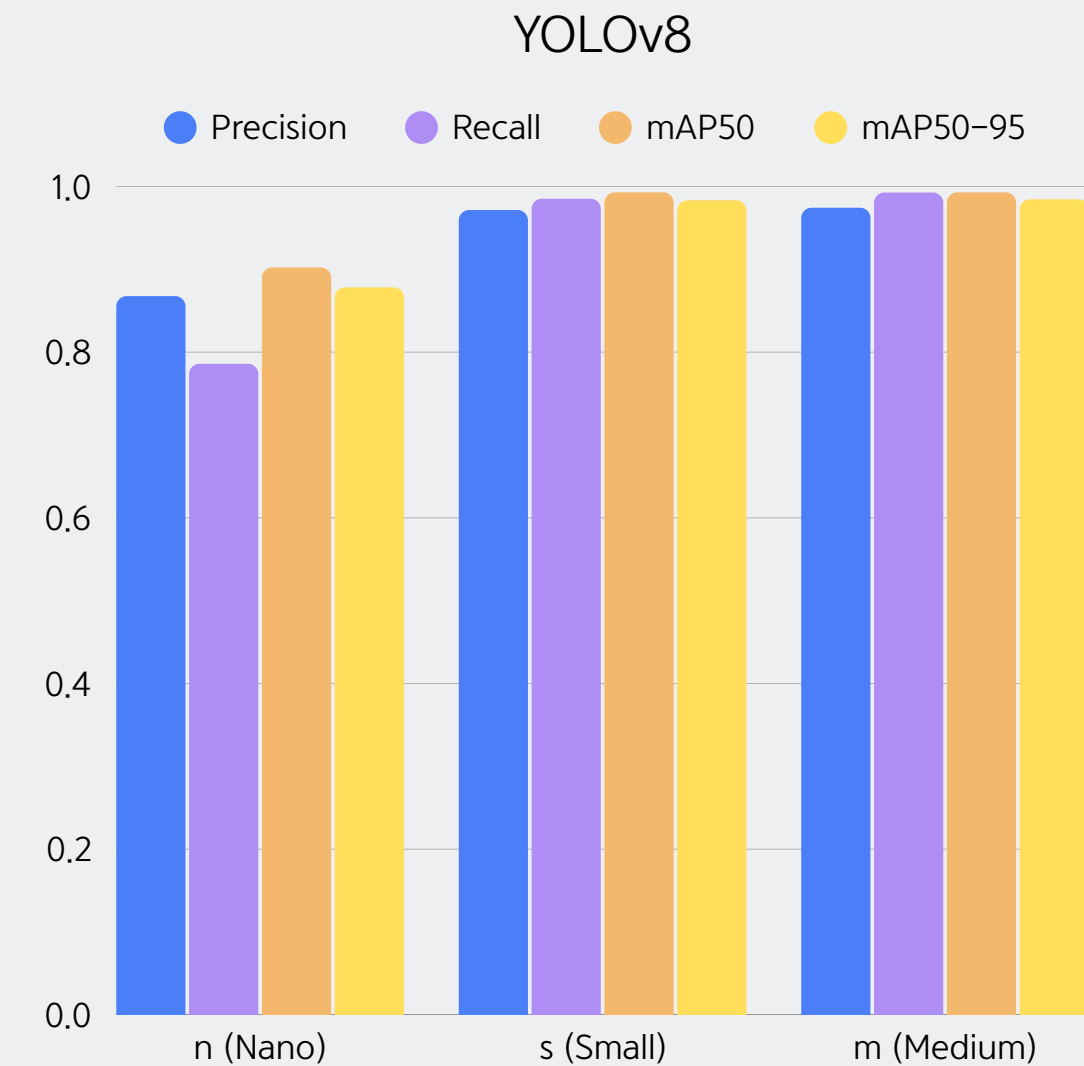
별도의 튜닝 없이도 mAP50-95에서 0.98 이상의 높은 성능을 기록함으로써, 데이터셋의 라벨링 상태가 양호하며 모델이 객체의 위치를 안정적으로 학습하고 있음을 증명함.

YOLOv8 체급 확장

모델 용량 증대에 따른 성능 변화 관찰을 위해 v8 체급별 비교 실험 진행함

Nano 대비 Small 체급에서 급격한 상승이 확인됨

이후 체급을 s로 확장하여 실험을 진행함

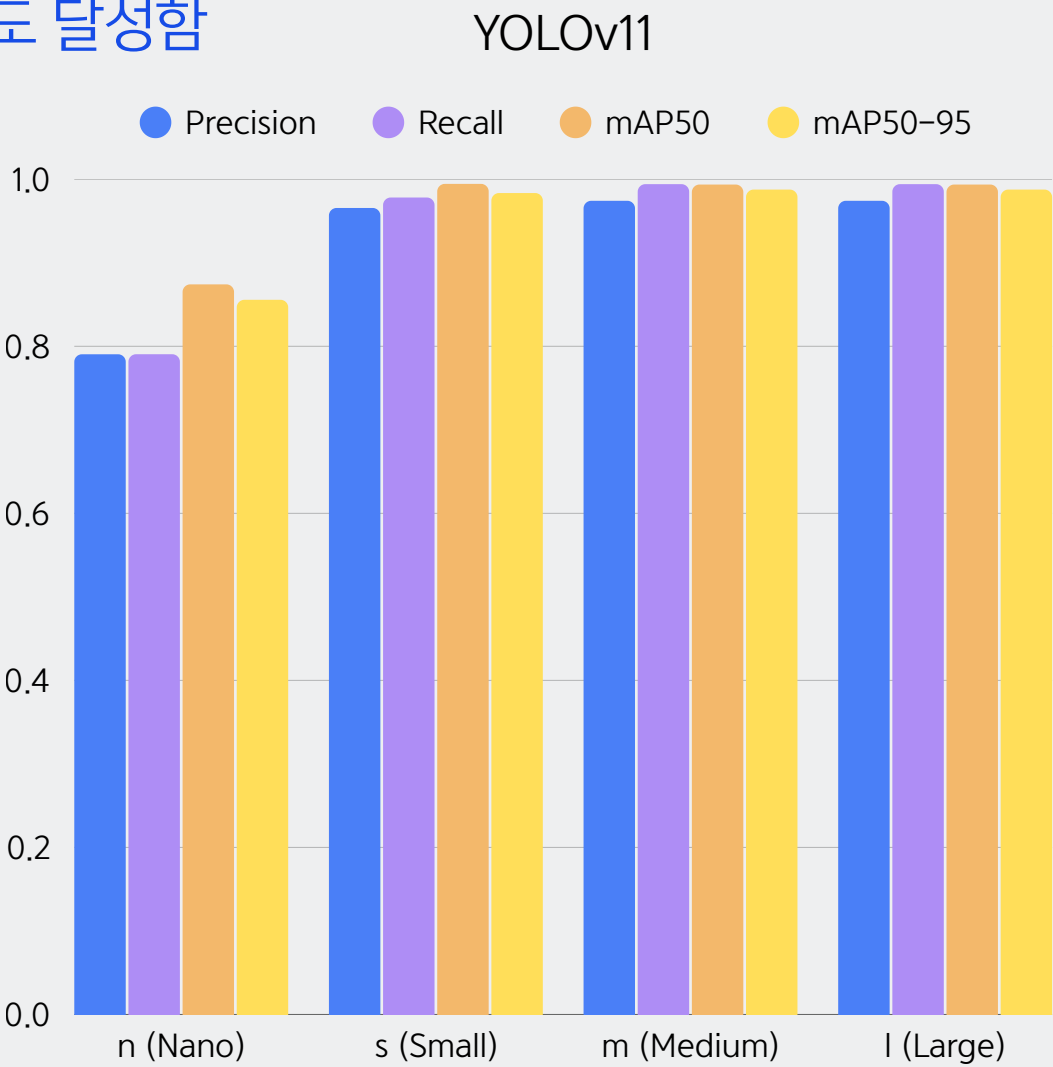


최적 아키텍처 전환 및 최적 체급 도출 (v8s → v11s)

YOLOv8 vs YOLOv11 차이

- YOLOv8: 가장 대중적이고, 다양한 환경에서 안정적인 성능 보장
- YOLOv11: PSA(어텐션) 탑재로 소형/복잡 객체 탐지력 극대화, v8 대비 적은 연산으로 고정밀도 달성함

비교 모델	선정 여부	주요 선정 사유
YOLOv8s	X	안정적이나 최신 아키텍처 대비 특징 추출력 열위
YOLOv11n	X	경량화에 치중되어 복잡한 클래스 식별력 부족
YOLOv11s	O	정확도(Accuracy)와 속도(Latency)의 최적 균형점 달성
YOLOv11m	X	성능 향상 폭 대비 연산 비용 과다 발생



베이스라인 2.0

1. Baseline 2.0 개요

- 모델 구조: **YOLO11s**
- 주요 설정:
 - 입력 해상도: 960
 - 전처리: 기본 이미지 (Raw Data)
 - 데이터 분할: Stratified Split (계층적 분할)
 - 증강(Augmentation): 희귀 클래스 증강 (Copy Paste)
- 핵심 지표: mAP50 기준 약 0.9704, 캐글 점수 약 0.96455

yolo11s

베이스라인 2.0

2. Baseline 선정 이유

(1) 모델 아키텍처의 성능 검증

약제의 미세한 각인이나 형태 차이를 구분해내는 **Precision**을 높이기 위해 더 깊은 레이어를 가진 모델 스케일을 선택함.

(2) 소수 클래스 탐지 성능 강화

특정 약제의 빈도가 낮은 '클래스 불균형' 문제를 해결하고자 Copy-Paste 증강을 적용함.

해상도	Precision	Recall	mAP50	mAP50-95	캐글점수
640	0.969	0.9861	0.9916	0.9892	0.96723
960	0.9704	0.9761	0.9918	0.9897	0.96455
1024	0.9692	0.9863	0.9929	0.99	0.97292
1280	0.9706	0.9634	0.9909	0.989	제출안함

학습 단계 최적화

학습 단계 최적화 CLAHE 적용

CLAHE(Contrast Limited Adaptive Histogram Equalization)란?

이미지의 대비를 향상시키는 기법 중 하나로, 일반적인 히스토그램 평탄화(HE)의 단점을 보완한 알고리즘



해상도640	Precision	Recall	mAP50	mAP50-95	캐글점수
clahe 적용	0.9768	0.9911	0.9948	0.9907	0.95902
clahe 미적용	0.9735	0.9914	0.9941	0.9872	0.95898

- 본 모델에서는 CLAHE를 통해 약제 각인 및 형태의 대비를 높여 성능 향상을 확인
- 단, Mosaic 및 HSV 변화가 적용된 환경에서는 CLAHE의 고정적인 대비 보정이 증강 효과를 상쇄하거나 노이즈를 유발할 수 있음을 확인하여, 실험을 통해 우리 모델에 가장 적합한 조합을 선택함

학습 단계 최적화 희귀 클래스 증강

[Problem] 전체 클래스 중 17개 클래스가 단 3개의 샘플만 보유
→ 학습 데이터 부족으로 인한 **희귀 알약의 탐지 성능 저하** 우려

[Strategy] 희귀 클래스 증강 기법 적용

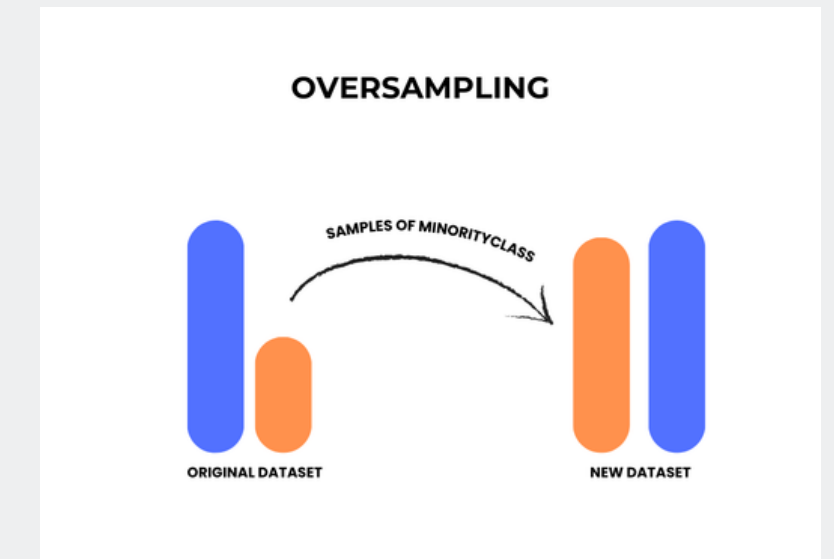
→ **OverSampling**: 단순 중복 활용

→ **Copy-Paste**: 알약 객체를 무작위 배경에 합성

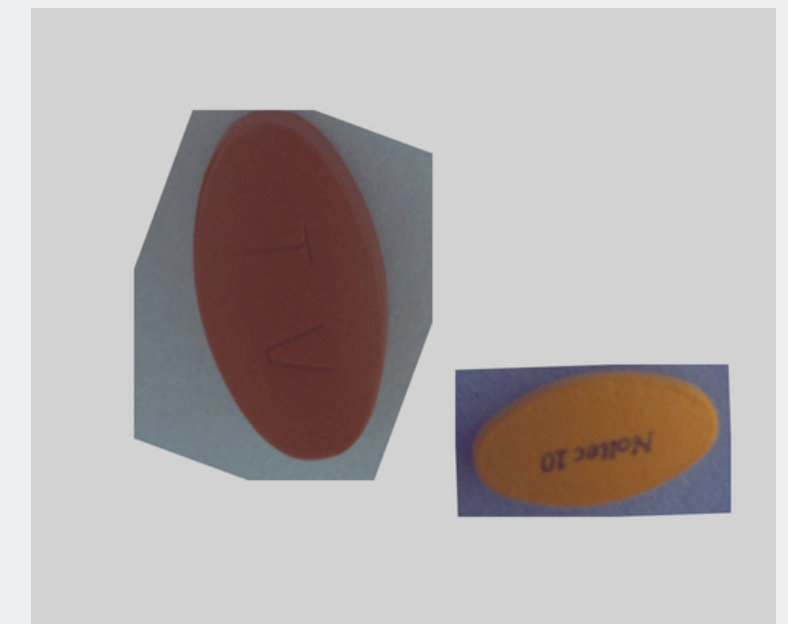
해상도640	Precision	Recall	mAP50	mAP50-95
미적용	0.9503	0.9805	0.979	0.9633
OverSampling	0.95203	0.97976	0.97373	0.96305
CopyPaste	0.9721	0.993	0.9931	0.9867

(미적용 캐글: 0.92235 → Copy-Paste 적용 캐글: **0.95967**)

실험 결과 모든 증강 기법 중 가장 압도적인 성능 향상 폭을 기록한 'Copy-Paste' 전략을 프로젝트 최우선 순위로 채택함



<OverSampling>



<Copy-Paste>

학습 단계 최적화 hsv_v 명도 튜닝

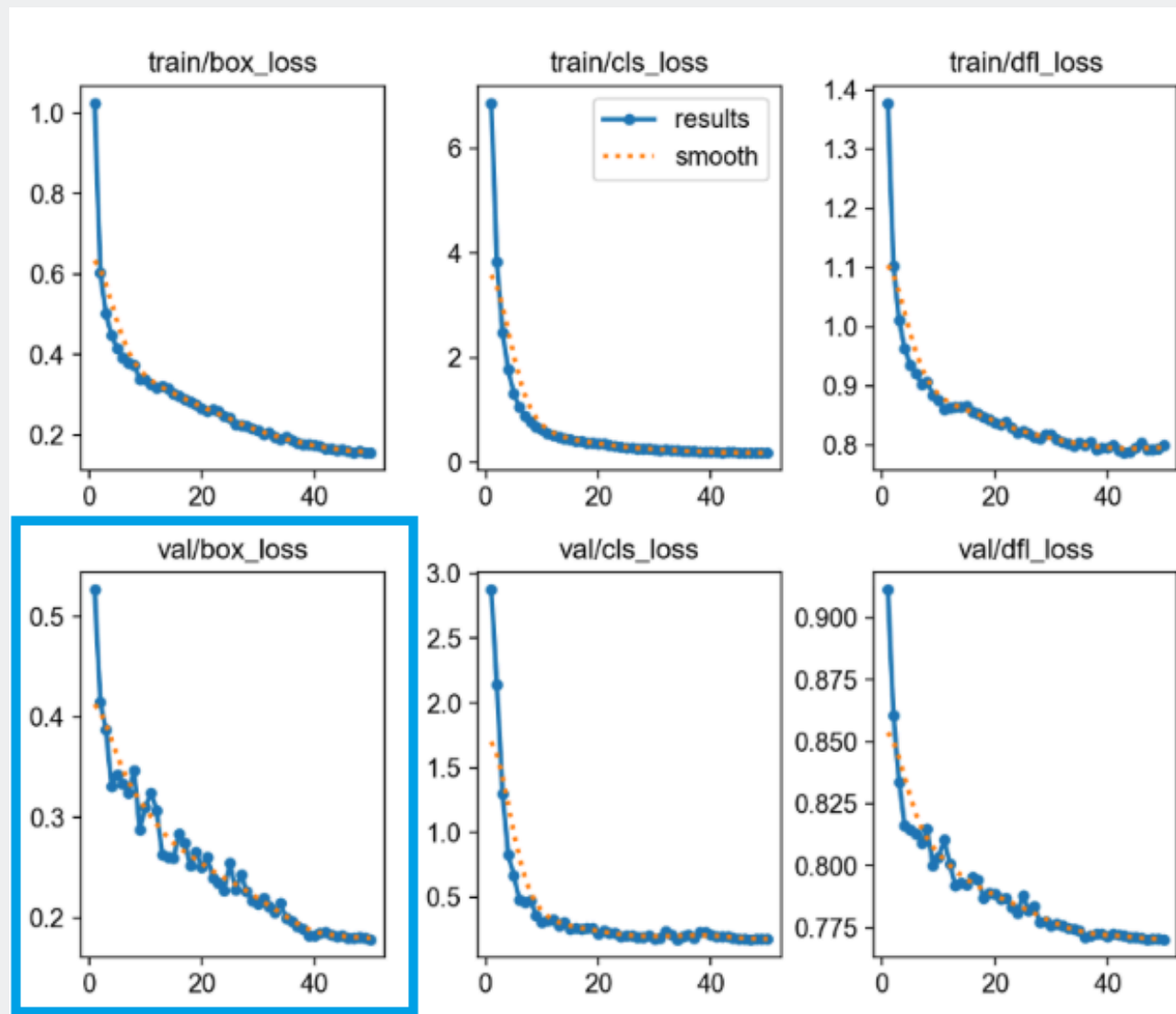
촬영 환경(조명, 그림자 등) 변화에 대한 모델의 강건성 확보를 위해 명도 변화 범위 최적화 진행(yolo11s, 해상도640)

hsv_v	mAP50-95	Kaggle Score	비고
0.4	0.9707	0.93634	리더보드 점수 우세
0.6	0.9741	0.92808	내부 지표 최고점 달성
0.8	0.9672	-	과도한 증강으로 성능 저하

YOLOv11s + Copy-Paste 기반 기초 실험

과도한 명도 증강(0.6 이상)은 조명 노이즈를 유발하여 오히려 실전 탐지력을 저하시키는 것으로 판단됨.
적절한 수준의 증강(0.4)이 테스트 데이터셋의 조명 환경과 가장 높은 유사성을 보이며 안정적인 성능을 유지함 확인됨.

학습 단계 최적화 Loss Weight 튜닝



- yolo11s, 해상도 640
- cls_loss: 학습 초기 급격한 하락 후 조기 안정화 양상을 보임
- box_loss & dfl_loss: 50 에폭까지 완만하지만 지속적인 하향 곡선을 그리며 수렴함
- 분석 결과: **위치 추적 정확도**의 추가 개선 여지가 존재함

(box, cls, dfl)	Precision	Recall	mAP50	mAP50-95
(7.5, 0.5, 1.5)	0.95605	0.97826	0.97716	0.96439
(8.5, 0.5, 1.5)	0.9444	0.9798	0.976	0.9664
baseline 2.0 (7.5, 0.5, 2.0)	0.9505	0.9807	0.9776	0.9581

- mAP50-95에서 유일하게 베이스라인에 비해 상승한 (8.5, 0.5, 1.5) 값이 가장 유의미함
- 다만, precision, recall, mAP50에서는 낮은 성능을 보이므로 추가 실험이 필요함

베이스라인 2.0 기반으로 재실험

단순 반복 실험이 아닌, 자원의 효율적 배분과 결과의 정밀도를 모두 잡기 위한 단계별 최적화 전략을 수립했음.

구분	Stage 1: 저해상도 탐색 (640)	Stage 2: 고해상도 검증 (960)
역할	효율적 가설 검증 및 후보군 압축	최종 성능 판단 및 실전 배포 기준
목적	빠른 학습 속도를 활용한 대규모 파라미터 탐색	실제 제출 환경에 최적화된 성능 도출
비고	유망 조합(Loss, Augmentation 등) 선별	선정된 후보군의 해상도 적응성 및 최종 점수 확정

학습 단계 최적화 Optimizer 튜닝

[목표] Yolo 11s에 자동 설정된 **AdamW**의 최적성 여부를 Adam계열(Adam, Adam W, RAdam 등)이 아닌 외 다른 계열의 옵티마이저: SGD로 우선 비교 검증

[검증] 모든 파라미터 통제, 옵티마이저만 SGD로 변경 vs 옵티마이저별 추천 세팅값과 함께 변경 Optimizer만 변경했을 때

- AdamW vs RAdam vs SGD → 동일 조건에서는 성능 차이 거의 없음
- AdamW 기준 lr 사용 시, SGD 급락: 학습 실패 수준 성능 (0.609) → **optimizer 자체보다 적절한 학습률 설정이 훨씬 중요**

Optimizer별 추천값 적용 시

- SGD (lr=0.01, momentum=0.937) → 성능 0.945까지 회복

Optimizer/해상도960	lr0	lrf	weight_decay	momenntum	Batch Size	캐글점수
ADamW (Auto)	0.000167	0.01	0.0005	0.9	16	0.96455
RAdam	0.000167	0.01	0.0005	0.9	4	0.91601
SGD (동일 설정)	0.000167	0.01	0.0005	0.9	16	0.6091
SGD + 추천값 (동일 설정)	0.01	-	-	0.937	16	0.94536

학습 단계 최적화 hsv_h 색상 튜닝

중간 강도 hue 증강의 효과

hsv_h=0.020은 색 변화에 대한 강건성을 확보하면서, 과도한 색 왜곡은 피한 값으로 해석됨 (베이스라인 2.0대비 **+0.00699**)

국소 최적점 존재

0.020 부근이 상위권이며, 0.030 구간은 하락 신호를 보여 과증강 리스크가 있음을 보여줌.

순위 (Kaggle)	실험명	hsv_h	Precision	Recall	mAP50	mAP50-95	Kaggle
1	Exp20-H3	0.02	0.9709	0.9667	0.9935	0.9919	0.97154
2	Exp19-A (No-HSVH)	0	0.9631	0.9642	0.9929	0.992	0.96678
3	Exp15-Baseline2.0	0.015	0.9704	0.9761	0.9918	0.9897	0.96455
-	Exp20-H2	0.01	0.9655	0.9649	0.9934	0.9915	미제출
-	Exp20-H7	0.025	0.9726	0.9749	0.9934	0.9911	미제출

학습 단계 최적화 Mosaic 튜닝

가설: mosaic 강도를 낮추면 실제 테스트 분포와 유사해져 캐글 기준 일반화 성능이 개선될 것임

결과: mosaic=0.00일 때 baseline 2.0 대비 **+0.00439**

해석: mosaic를 줄일수록(특히 0.0) 실제 테스트 분포와 정합성이 높아져 Kaggle 일반화 성능이 개선됨.
반대로 mosaic=0.6은 로컬 mAP는 높아도 비현실적 합성 패턴 영향으로 Kaggle 점수가 하락함.

향후: hsv_h(0.02) + mosaic(0.0) 조합 을 기준으로 삼음

순위 (Kaggle)	실험명	mosaic	Precision	Recall	mAP50	mAP50-95	Kaggle
1	Exp21-M0	0	0.9659	0.9707	0.9945	0.9916	0.96894
2	Exp15-Baseline2.0	1	0.9704	0.9761	0.9918	0.9897	0.96455
3	Exp21-M2	0.6	0.9747	0.9648	0.9939	0.992	0.96154
-	Exp21-M1	0.4	0.9727	0.9622	0.9918	0.9897	미제출

학습 단계 최적화 CLAHE 튜닝

가설: Exp22-H3M0(hsv_h=0.02, mosaic=0.0) 실험에서 CLAHE변형을 추가하면 조명/대비 변화에 대한 강건성이 올라갈 것임

결과: baseline2.0 대비 C1은 거의 동일(**-0.00020**), C2는 큰 폭으로 하락(**-0.01659**).

해석: 현재 데이터 분포 + 기존 증강(hsv_h/mosaic) + CLAHE 조합에서 **과보정**이 발생
CLAHE 강도가 올라갈수록(특히 **cliplimit=2.0**) 대비가 과하게 증폭되어 실전 분포에서 오탐/분포 왜곡이 커진 것으로 해석됨
저해상도에서 유효할 수 있는 CLAHE 개선 전략이 적용되지 않았음

순위 (Kaggle)	실험명	clipLimit	tileGridSize	hsv_h	Precision	Recall	mAP50	mAP50-95	Kaggle
1	Exp22-H3M0 (기준)	-	-	0.02	0.9753	0.963	0.9935	0.9915	0.97199
2	Exp15- Baseline2.0	-	-	0.015	0.9704	0.9761	0.9918	0.9897	0.96455
3	Exp23-C1	1.5	(8,8)	0.02	0.9683	0.9643	0.9948	0.9941	0.96435
4	Exp23-C2	2	(8,8)	0.02	0.9548	0.9801	0.9931	0.9914	0.94796
-	Exp23-C3	2.5	(8,8)	0.02	0.9347	0.9926	0.9946	0.9928	미제출

학습 단계 최적화 hsv_v 명도 튜닝

가설: hsv_v(명도 변형) 조정으로 조명 변화에 대한 강건성이 높아질 것임

결과: 로컬 mAP는 일부 개선 신호가 있었지만, 적용 결과 기준 실험보다 **-0.00891** 하락

해석: 해상도960 + hsv_h + mosaic 조합 환경에서는 추가 명도 변형이 오히려 분포 왜곡을 유발할 가능성

순위(Kaggle)	실험명	hsv_v	Precision	Recall	mAP50	mAP50-95	Kaggle
1	Exp22-H3M0 (기준)	0.4	0.9753	0.963	0.9935	0.9915	0.97199
2	Exp24-V3	0.5	0.96	0.9818	0.9945	0.9922	0.97016
2	Exp15-Baseline2.0	-	0.9704	0.9761	0.9918	0.9897	0.96455
4	Exp24-V2	0.2	0.9559	0.9854	0.9943	0.9928	0.96308
-	Exp24-V1	0	0.9749	0.9641	0.9935	0.9906	미제출

학습 단계 최적화 Loss weight 튜닝

가설: box/cls/df1 gain 조정을 통해 기준 실험(hsv_h+mosaic) 대비 성능 향상이 있을 것임

로컬 결과: 2번째 실험(LBD1)이 mAP50-95 **+0.0008** 로 최고

실전 결과: Kaggle은 기준 실험보다 **- 0.00269** 낮은 수치

결론: Loss gain 튜닝은 해상도 960에서 최적값 찾지 못함, 다른 실험인 추론 최적화(WBF)를 진행하는 것이 시간상 이득

순위 (Kaggle)	실험명	box, cls, dfl	Precision	Recall	mAP50	mAP50-95	Exp22-H3M0와 비교	Kaggle
1	Exp22-H3M0 (기준)	(7.5, 0.5, 1.5)	0.9753	0.963	0.9935	0.9915	0	0.97199
2	Exp25-LBD1	(8.5, 0.5, 2.0)	0.9741	0.9856	0.9941	0.9923	0.0008	0.9693 
3	Exp15- Baseline2.0	(7.5, 0.5, 1.5)	0.9704	0.9761	0.9918	0.9897	-0.0018	0.96455
-	Exp25-LD1	(7.5, 0.5, 2.0)	0.9787	0.9599	0.9944	0.992	0.0005	미제출
-	Exp25-LB1	(8.5, 0.5, 1.5)	0.973	0.9632	0.9932	0.9919	0.0004	미제출

추론 성능 극대화

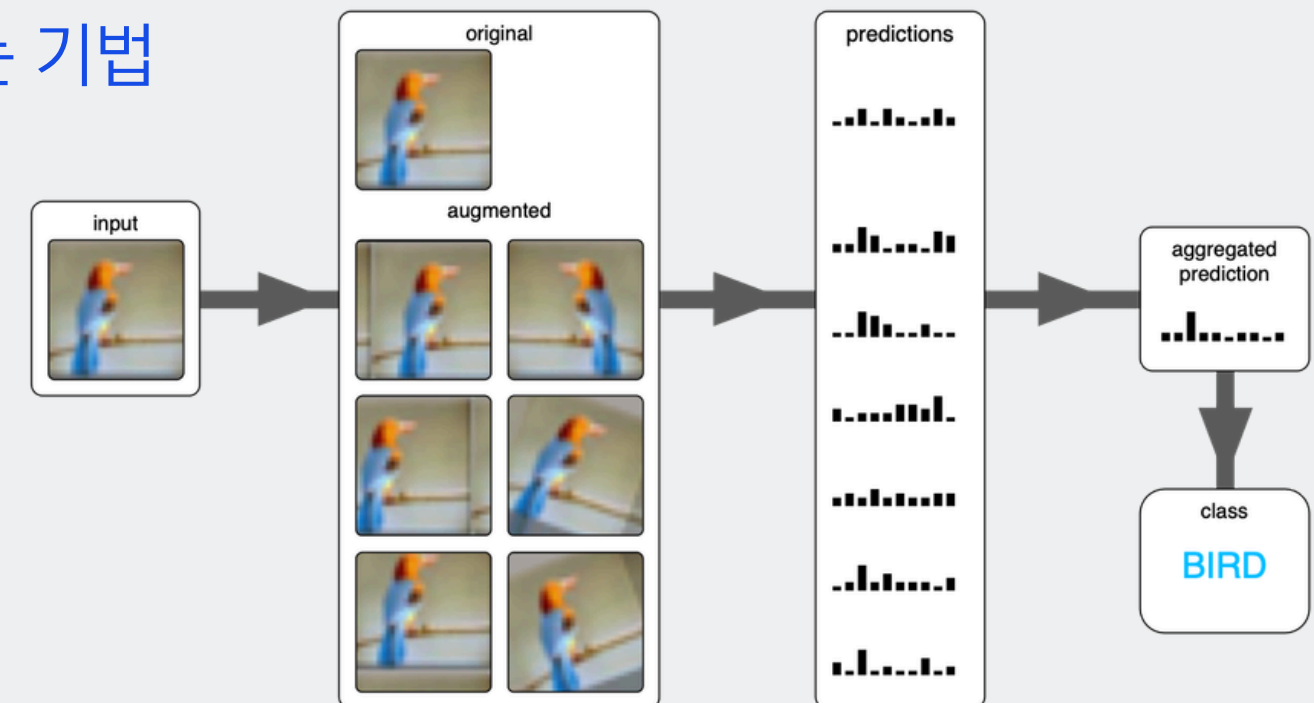
추론 성능 극대화

TTA(Test Time Augmentation)

추론 단계에서 입력 이미지를 여러 방식으로 변형하여 예측 성능을 높이는 기법

- 적용 변형
 - Flip
 - Multi-scale

	캐글점수
TTA 적용	0.94215
TTA 미적용	0.97199



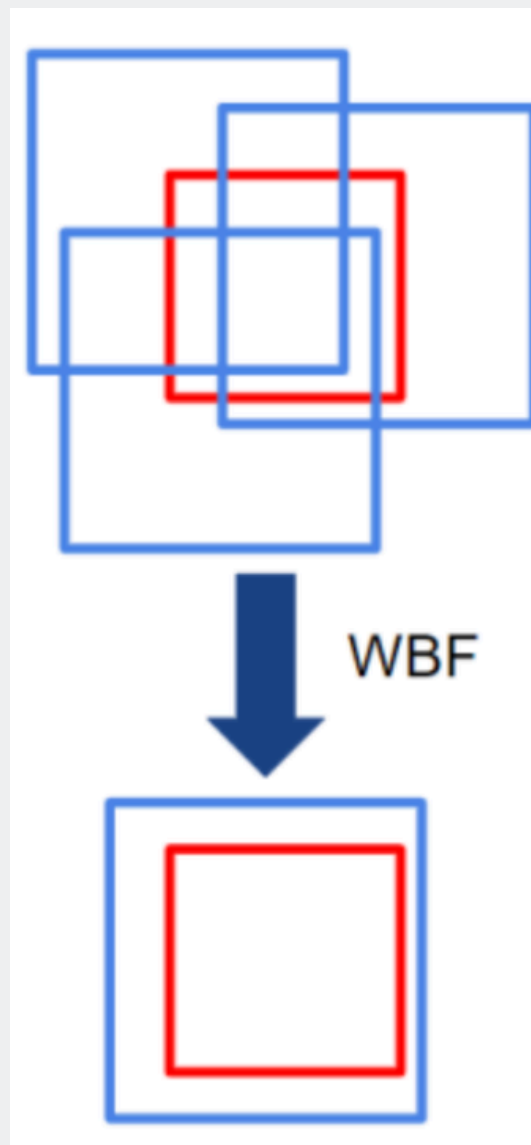
추론 성능 향상을 위해 TTA를 도입하였으나, 미적용 시보다 캐글 점수가 하락하는 결과를 보임

단순히 여러 장을 보는 것이 항상 정확도 향상으로 이어지지 않음을 확인하고, 본 프로젝트 데이터셋과의 부적합성을 분석함
Multi-scale 적용에 따른 미세 특징 정보 왜곡 및 박스 통합 과정의 좌표 오차 발생이 성능 하락의 주요인으로 분석됨

추론 성능 극대화

WBF (Weighted Boxes Fusion)

겹치는 모든 박스의 정보를 가중 평균하여 새로운 박스를 만드는 방식



실무에서 중요한 강건성 + 재현성 + 운영안정성 근거를 같이 확인

- 1차(다중해상도): 입력 해상도 변화에 대한 모델 민감도 점검
- 2차(다중시드): 동일 seed 설정에서 변동 리스크 완화 및 결과 안정성 확인

- 특징: 신뢰도가 높은 박스에는 높은 가중치를, 낮은 박스에는 낮은 가중치를 주어 좌표 계산.
- 장점: 여러 예측값의 "**평균**"을 내기 때문에, 단일 박스보다 실제 객체의 위치에 훨씬 더 **가깝게** 박스가 형성됨.

구분	실험명	Precision	Recall	mAP50	mAP50-95	Kaggle
기준	Exp22-H3M0	0.9753	0.963	0.9935	0.9915	0.97199
1차 WBF	Exp26 (Exp22 + 640/960/1024 WBF)	-	-	-	-	0.97345
2차 WBF	Exp27 (42/123/777 WBF)	-	-	-	-	0.98077

결론 및 향후 과제

결론

1. 실험을 통한 최적의 아키텍처 및 설정 도출

- **모델 체급의 황금 밸런스: YOLO11s** 모델임에도 불구하고 캐글 0.98077이라는 고득점을 달성하며, 실전 성능과 실험 효율의 최적점을 찾아 최적의 모델 효율을 증명함.
- **해상도 전략**
 - 1차로 640에서 빠르게 후보를 탐색하고, 2차로 선별된 조합을 960에서 재검증하는 **2-Stage 전략**을 적용
 - 제출 기준 해상도는 팀 공통 재현성과 반복 검증 안정성을 고려해 960으로 통일하고, 최종 성능 개선은 추론 최적화(WBF/TTA)로 보완

2. 데이터 중심 최적화의 유효성

- **라벨링 정제의 힘:** 9장의 Bounding Box 오류를 수동으로 JSON 좌표 보정한 결과, 데이터 손실 없이 학습 데이터셋의 완전성을 확보하여 모델의 신뢰도를 높였음.
- **증강 기법의 명암:** Copy-Paste와 hsv_h 조정은 희귀 클래스 학습과 색상 변화 대응에 유효했으나, hsv_v와 CLAHE 실험을 통해 과도한 전처리가 오히려 실전 환경에서의 일반화 성능을 저해할 수 있다는 '과증강 리스크'를 실증했음.

향후 과제

1. MLOps 환경 구축 및 실험 관리 자동화

- **실험 트래킹 도입:** 많은 실험을 관리하며 겪은 어려움 → WandB나 MLflow를 도입
- **데이터 및 모델 버전 관리(DVC):** 언제든지 특정 시점의 실험을 복구하고 협업할 수 있는 엔지니어링 파이프라인을 고도화할 예정



2. 엄격한 재현성 검증 체계 구축

- **실행 환경 제어 강화:**

실험 과정에서 발생했던 '동일 설정, 타 성능' 문제를 해결하기 위해, CUBLAS_WORKSPACE_CONFIG 설정을 환경변수로 고정해 GPU 연산의 재현성을 강화할 예정임

- **통계적 유의성 확보:**

Multi-seed(3~5회) 반복 실험을 통한 평균 성능 및 표준편차를 기록하여 모델의 신뢰도를 객관화할 계획임.

Thank you!

PillaTech (4팀)의 발표를 들어주셔서 감사합니다.